📸 OpenSpace Capture Upload API – Private Integration

This document describes how to manually upload capture data to OpenSpace via our private API. This workflow is designed for integrations that cannot use our native applications.

📸 Camera Setup

Use Correct Camera

Captures should be taken using an Insta360 OneX3, OneX4, or OneX5 camera.

Configure Camera

Before capturing, ensure the camera is configured using the following settings:

```None
      Mode: Timelapse
   Interval: 0.5 sec
    Bitrate: High
   Exposure: Auto
Resolution: 5.7k
```

To set the camera bitrate:

1. Access the Settings: Swipe down on the Insta360 X4's screen to open the settings menu.
2. Navigate to Image Settings: Tap the settings cog icon and scroll down to "Image Settings".
3. Set the Bitrate to “High”

🔐 Authentication

Get Your Credentials

Before you begin, you will need the following credentials. The link to this document should have come with a 1Password link as well.

- `client_id`: Your integration's tenant ID (will be shared with you via 1Password)
- `client_secret`: Your client secret (will be shared with you via 1Password)
- `username`: OpenSpace user email
- `password`: OpenSpace password (you will need to set this yourself)

The account tied to the username (an email) and password needs at least "Editor" permission on the OpenSpace site you're capturing on. You will probably want this to be a machine user account that is only used for this and is not shared with a human using OpenSpace normally.

Request an `authToken`

The login hostname is the same regardless of your production region domain. Use the following `curl` command to authenticate:

```shell
curl --location 'https://login.openspace.ai/oauth/token' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'audience=openspace.ai' \
--data-urlencode 'client_id={yourTenantId}' \
--data-urlencode 'client_secret={yourSecret}' \
--data-urlencode 'username={yourUsernameEmail}' \
--data-urlencode 'password={yourUsernamePassword}'
```

This will return:

```json
{
  "access_token": "eyJ...yourJWT...",
  "expires_in": 86400,
  "token_type": "Bearer"
}
```

Include this `access_token` in all subsequent requests:

```None
Authorization: Bearer {authToken}
```

📄 Terminology

Term	Description
`authToken`	A JWT used for authenticated requests.
`url-id`	A base64 and URL-encoded UUIDv4. Must be unique. For example, to generate on Mac: Shell `uuidgen | xxd -r -p | base64 |tr '/+' '_-' | tr -d '='` Will give you something like "VgI6T37STOS57d5H65u9og"
`sheetId`	Unique identifier for the floor plan being captured. (A `url-id`)
`siteId`	Unique identifier for the project. (A `url-id`). This is in most URLs when using the webapp. We will provide it directly for your test site.
`captureName`	Your chosen name for the capture.
`startMicro`	The capture start time as a Unix timestamp in microseconds.
`aspectRatio`	Sheet width-to-height ratio (e.g., 2 = twice as wide as tall).
`startPosition`	`[x, y, z]` coordinate on the floor plan. • [0]: the x coordinate on the floorPlan. -1 is all the way to the left. 1 is all the way to the right. 0 is the center • [1]: the y coordinate on the floorPlan. \`-1/aspectRatio\` is all the way to the bottom. \`1/aspectRatio\` is all the way to the top. 0 is the center • [2]: the z coordinate on the floorPlan. For your purposes, you can just set this to 1.5, always

`deviceId`	Identifier for the camera used. Format: `{CameraType}:sn:{SerialNumber}`
	
	For example:
	
	None
	`Insta360 OneX2:sn:IXSEXXXXXXXXX2`
	`Insta360 OneX3:sn:IXSEXXXXXXXXX3`

```
Insta360 OneX4:sn:IXSEXXXXXXXXX4
```

`timezoneOffsetMinutes`	Offset from UTC in minutes.
`tags`	Labels for the upload. Include `"manual"` to mark this as non-app generated.
`uploadId`	Unique file identifier (usually a `url-id` with the file extension appended).

🧭 Workflow Overview

We are not including a domain because we operate in multiple regions, each with its own domain. Our default/US is https://openspace.ai, but use the OpenSpace domain you use for your project.

✅ Step 1: Create the Capture

Endpoint:
`POST /api/upcap/site/{siteId}/upcap-session`
Headers:
`Authorization: Bearer {authToken}`
Body:

```json
{
  "id": "3XkqBrPZQkGJ-Ryq4AkVMg",
  "sheetId": "OyYhWXVYQyOTdayOLsSkHA",
  "captureName": "Robot Walk",
  "startMicro": 1744152543953000,
  "startPosition": [0.35, -0.19, 1.5],
  "deviceId": "Insta360 X3:sn:ISXECAFEBEEF",
  "deviceState": {
    "timezoneOffsetMinutes": -300
  }
```

```
}
```

- You must generate the `id` (a `url-id`) yourself.
- The `startPosition` can default to `[0, 0, 1.5]` if unsure.

📎 Step 2: Attach Capture Metadata

Endpoint:
`POST /api/upcap/site/{siteId}/session/{captureId}/match`
Headers:
`Authorization: Bearer {authToken}`
Body:

```json
{
  "startMicro": 1744152543953000,
  "files": [
    {
      "deviceId": "Insta360 X3:sn:ISXECAFEBEEF",
      "deviceUrl": "VID_20250408_123000_00_123.insv",
      "file": {
        "id": "l-GXILlTQqiTCVJZWGmhyw.insv"
      }
    }
  ]
}
```

- `deviceUrl`: The filename as seen on the SD card.
- `file.id`: Your generated `uploadId`, formatted like a filename.

🗂️ Step 3: Register the Upload

Endpoint:
`POST /api/site/{siteId}/upcap/uploads`
Headers:
`Authorization: Bearer {authToken}`
Body:

```JSON
{
  "deviceId": "Insta360 X3:sn:ISXECAFEBEEF",
  "deviceFilename": "VID_20250408_123000_00_123.insv",
  "tags": ["manual"],
  "contentType": "video/insv",
  "size": 20973651,
  "numParts": 1,
  "uploadId": "l-GXILlTQqiTCVJZWGmhyw.insv"
}
```

Important:

- `deviceFilename` **must match** the actual filename on the camera.
- `size` must be the exact size (in bytes) of the file.
- `numParts` = `1` unless doing a multi-part upload (not supported in this doc).
- You must generate the `uploadId` yourself.

🆙 Step 4: Upload the File

Endpoint:
`PUT /api/site/{siteId}/upcap/uploads/{uploadId}?partNum=1`
Headers:

```None
Authorization: Bearer {authToken}
Content-Type: video/insv
Content-Range: 0-{size-1}/{size}
```

Body:
Binary file content

✅ You're uploading a full file in one go here so `Content-Range` will always be: `0-{size-1}/{size}`

Example:

```None
Content-Range: 0-20973650/20973651
```

⏳ Step 5: Wait for Processing

Endpoint:
`GET /api/site/{siteId}/layout2?full=true`
Headers:
`Authorization: Bearer {authToken}`

Check the `pendingCaptures` field in the response:

```JSON
{
  "pendingCaptures": [
    {
      "clientCaptureId": "3XkqBrPZQkGJ-Ryq4AkVMg",
      "submitted": false
    }
  ]
}
```

When the capture disappears from `pendingCaptures`, it has finished processing and will be available in the webapp shortly. This will be a function of the length of the capture, how man captures have been on that sheet, and the quality of the capture 🎉

✅ You're Done

Once your capture is processed and visible in the webapp, you're all set.